Machine Learning

Linear Regression

Dong-Kyu Chae

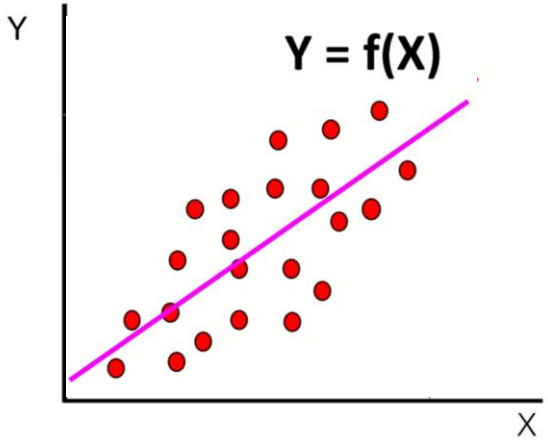
PI of the Data Intelligence Lab @HYU
Department of Computer Science
Hanyang University

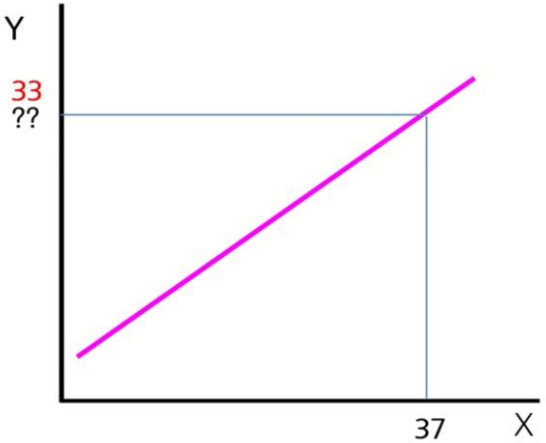
Data
Intelligence
LAB

Linear Regression Example

❑ Predicting price of houses

관측치	Area, 집크기 (X)	Appraised Value, 집가격 (Y)
1	1380	76
2	3120	216
3	3520	238
4	1130	69
5	1030	50
6	1720	119
7	3920	282
8	1490	81
9	1860	132
10	3430	228
11	2000	145
12	3660	251
13	2500	170
14	1220	71
15	1390	29





Multiple Linear Regression

- **Example: Price of old cars (multiple input variables)**

Target variable

Input features

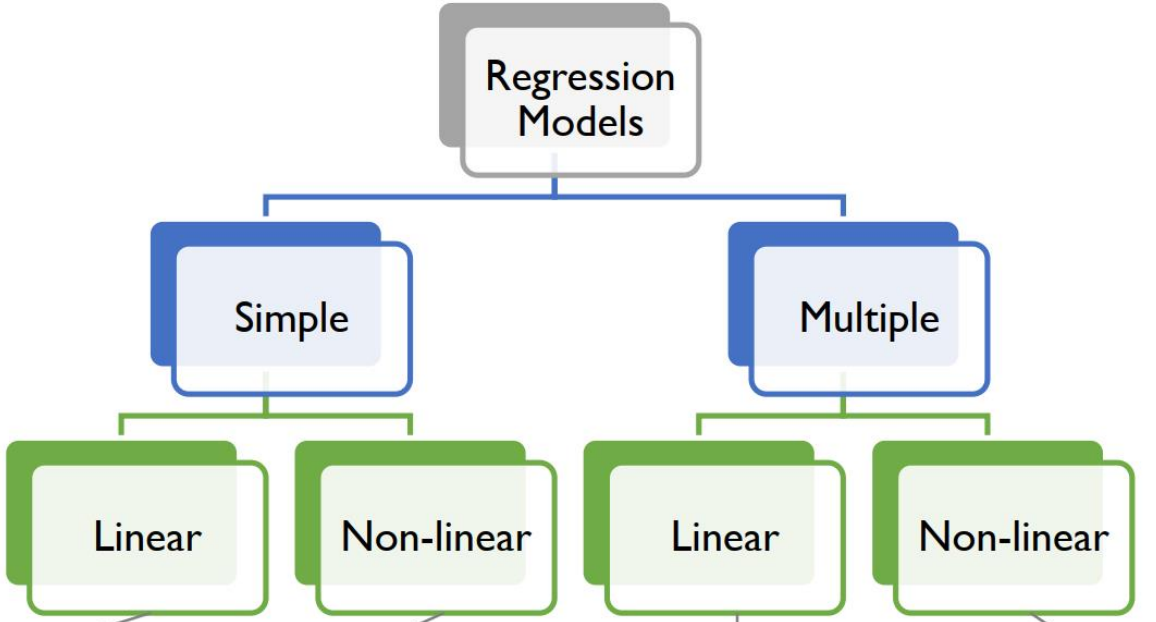
Variable	Description
Price	Offer Price in EUROs
Age_08_04	Age in months as in August 2004
KM	Accumulated Kilometers on odometer
Fuel_Type	Fuel Type (Petrol, Diesel, CNG)
HP	Horse Power
Met_Color	Metallic Color? (Yes=1, No=0)
Automatic	Automatic ((Yes=1, No=0)
CC	Cylinder Volume in cubic centimeters
Doors	Number of doors
Quarterly_Tax	Quarterly road tax in EUROs
Weight	Weight in Kilograms

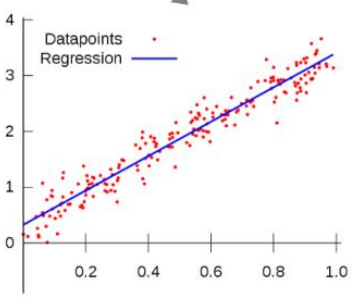
What Is Regression?

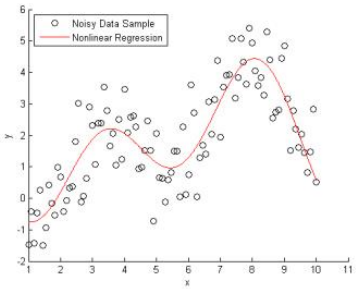
- **Regression**
  - Models the relationship between one or more predictor variables (i.e., **independent variables**) and a response variable (i.e., **dependent variable**)

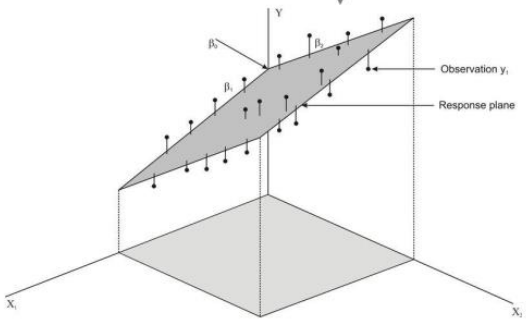
- **Types of regression**
  - Linear regression and **multiple linear regression**
  - Non-linear regression
  - Other regression methods
    - Generalized linear model, Poisson regression, log-linear models, regression trees, ….

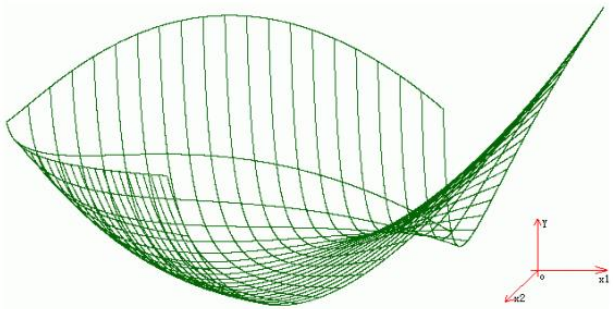
Types of Regression











Simple Linear Regression

- **(Simple) Linear regression:**
  - Involves a dependent variable **y** and a single independent variable **x** (linear function)

    -> $y = w_0 + w_1 x$

    where w_0 (y-intercept) and w_1 (slope) are regression **coefficients**
  - Training

    To estimates the best-fitting straight line

    $$\mathbf{y = w_0 + w_1 x}$$

    w_0 and w_1 need to be estimated by using the training data

Multiple Linear Regression

Multiple linear regression

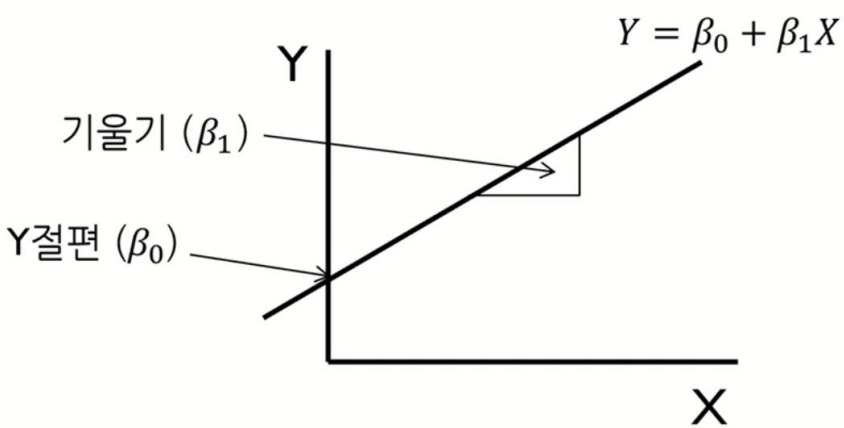
- Involves more than one independent/input variable
  - $X = <x_1, x_2, x_3, \ldots, x_n>$: input variables
  - y: dependent/output/target variable
- Training data is composed of the form $(\mathbf{X_1}, y_1), (\mathbf{X_2}, y_2), \ldots, (\mathbf{X_{|D|}}, y_{|D|})$
- Ex. For 2-dimensional data, we may have: $y = w_0 + w_1 x_1 + w_2 x_2$

Linear Regression

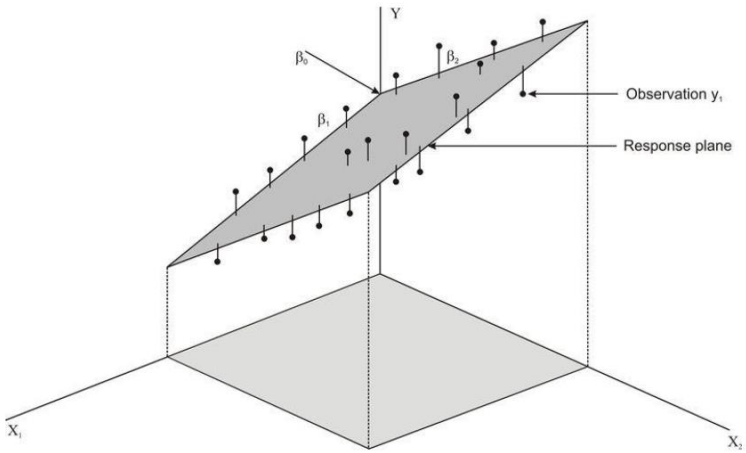
Idea

- Assume that the relationship between the input variables and the target variable is always **linear**

- Fit a linear relationship between a **target variable Y** and a set of **input features** $x_1, x_2, \ldots, x_d$ by: $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 \cdots + \hat{\beta}_d x_d$
  - Training: to find the proper set of coefficients $\hat{\beta}$



< Single input >

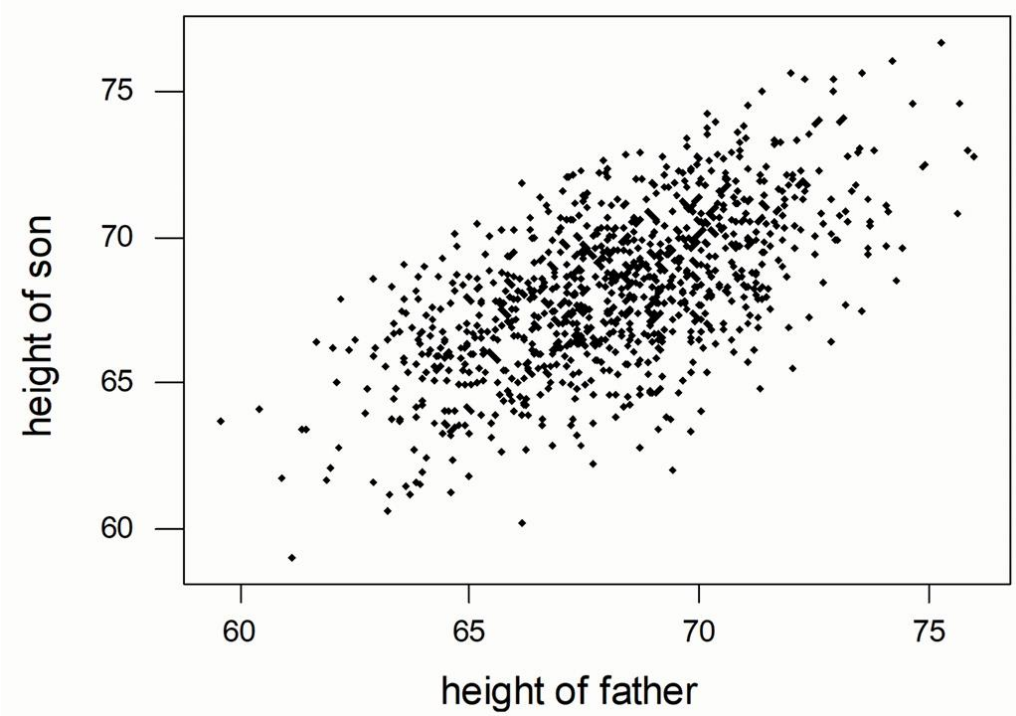


< Multiple inputs >

Linear Regression

- **Which model is the best?**
  - How to get the optimal $\hat{\beta}$?

$$E(Y) = f(X) = \beta_0 + \beta_1 X_1$$



Model Training

1. **Define a cost (or, loss, error) function $\mathbf{E}$**
   - **Difference** between the model's predictions and actual values

2. **Solve the optimization problem**
   - Getting optimal $\hat{\beta}$ that minimizes the error function $\mathbf{E}$

- **Squared error function**

Actual target: y

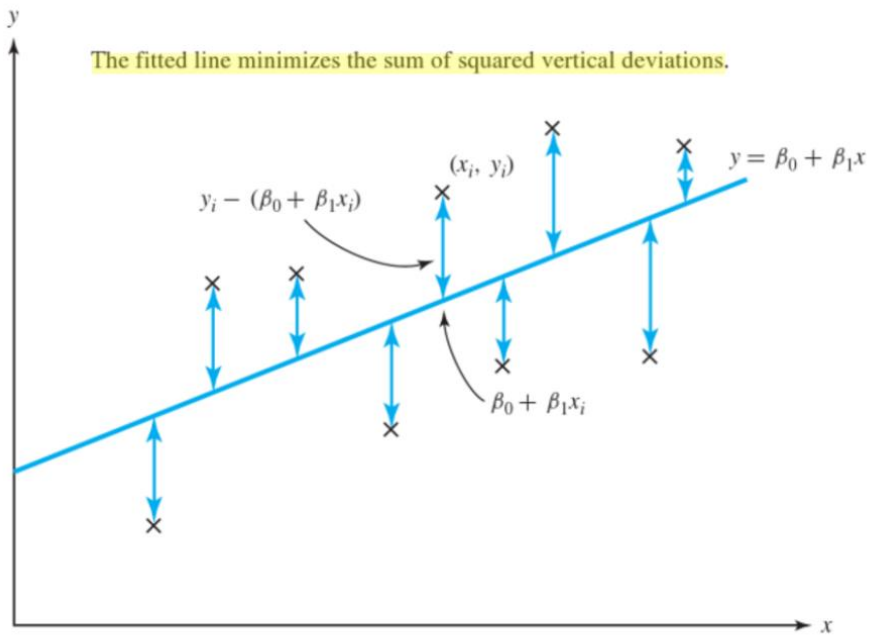
Predicted target: $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 \cdots + \hat{\beta}_d x_d$

$$\underset{\beta}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

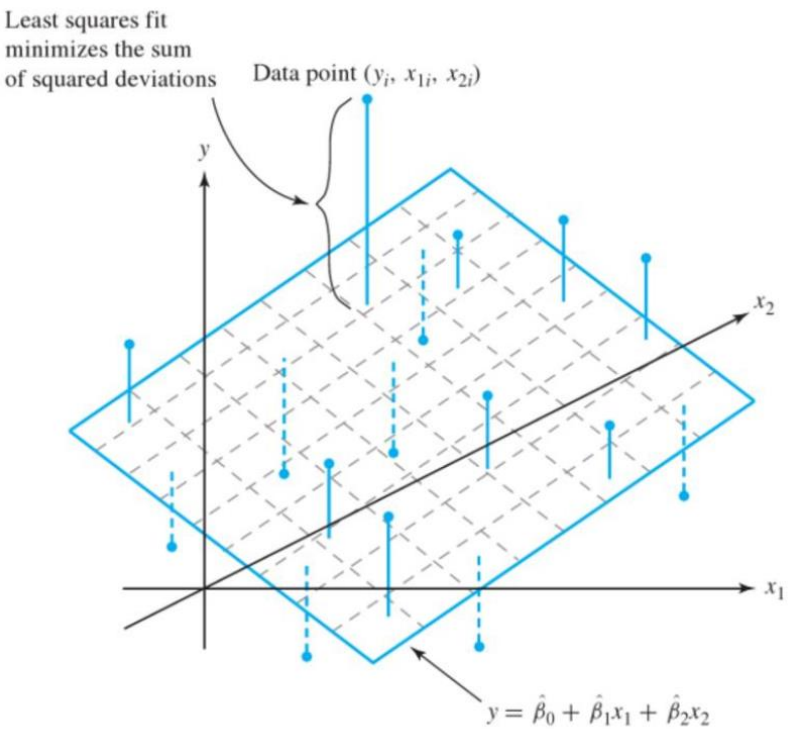
$$= \frac{1}{2} (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} \cdots + \hat{\beta}_d x_{id})^2 \ )$$

Model Training

❑ Squared error: graphical view



< Single input >



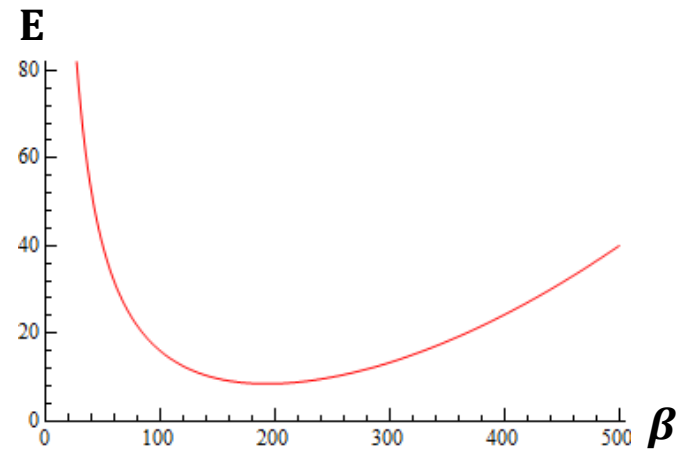
< Multiple inputs >

Model Training

How to solve the optimization problem?

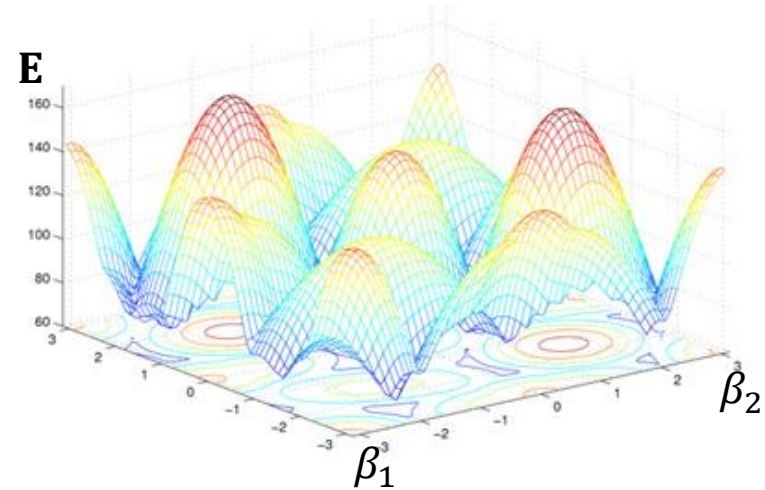
- If the cost function **E** is simple:
  - You can directly find the optimal parameters via computing the points where the **derivative** of E becomes zero

$$\frac{\partial E}{\partial \beta} = 0$$



- If the cost function **E** is **complex**:
  - Multiple model parameters…
  - Model parameters are aligned with each other…
  - We cannot directly solve the problem via

$$\frac{\partial E}{\partial \beta} = 0$$



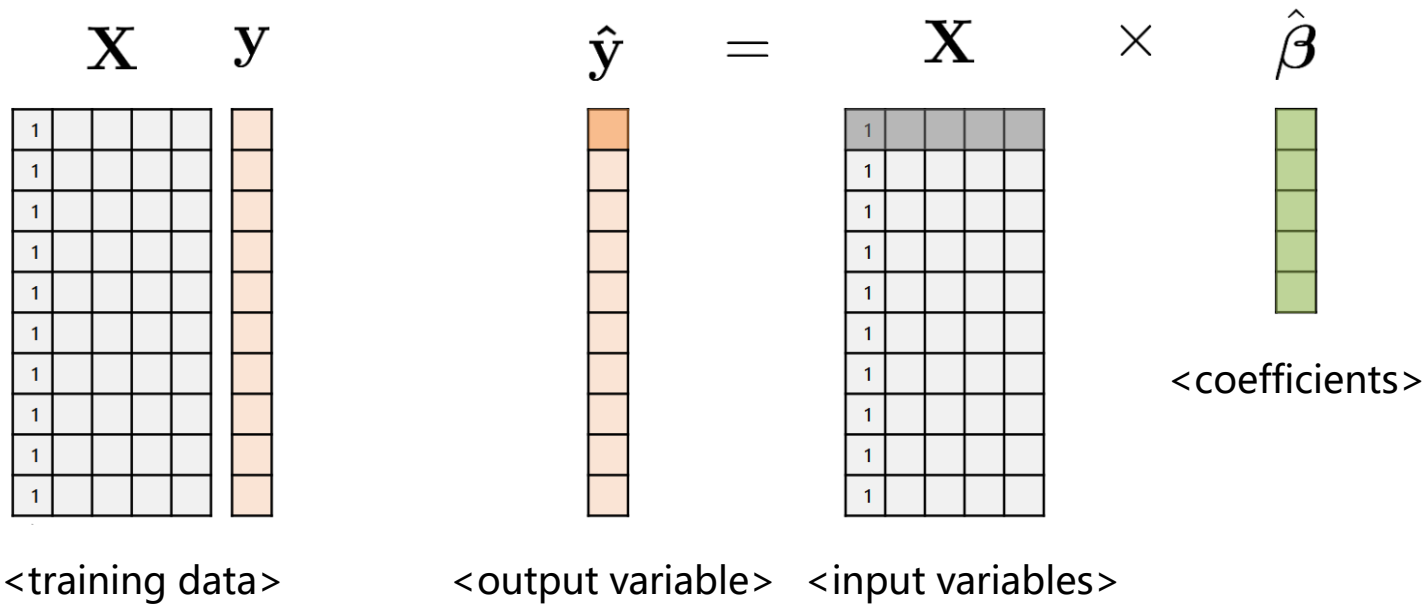
Model Training

Prepare variables and model parameters

- We will do it via matrix computation

$$\mathbf{X}: \ n \times (d+1) \ matrix, \ \mathbf{y}: \ n \times 1 \ vector$$

$$\hat{\boldsymbol{\beta}}: (d+1) \times 1 \ vector$$



Linear Regression

Defining loss function

$$\mathbf{X}: \ n \times (d+1) \ matrix, \ \mathbf{y}: \ n \times 1 \ vector$$

$$\hat{\boldsymbol{\beta}}: (d+1) \times 1 \ vector$$

$$\underset{\beta}{\operatorname{argmin}} \, E(\mathbf{X}) = \frac{1}{2}\left(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}\right)^T \left(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}\right)$$

$$\Rightarrow \frac{\partial E(\mathbf{X})}{\partial \hat{\boldsymbol{\beta}}} = -\mathbf{X}^T\left(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}\right) = 0$$

$$\Rightarrow -\mathbf{X}^T\mathbf{y} + \mathbf{X}^T\mathbf{X}\hat{\boldsymbol{\beta}} = 0$$

$$\hat{\boldsymbol{\beta}} = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y} \longrightarrow$$ Unique and explicit solution exists!

Linear Regression

- **(Appendix) Common matrix derivative formulas**

$$\frac{\partial(AX)}{\partial X} = A^T$$

$$\frac{\partial(X^T A)}{\partial X} = A$$

$$\frac{\partial(X^T X)}{\partial X} = 2X$$

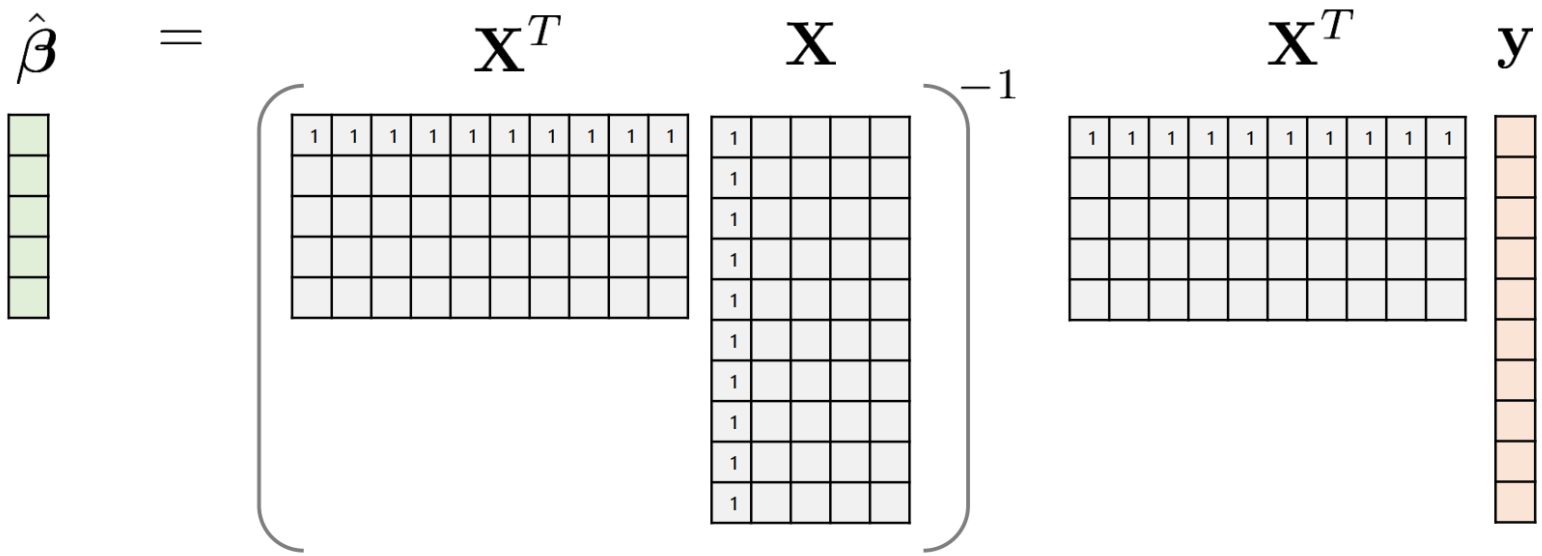
$$\frac{\partial(X^T A X)}{\partial X} = AX + A^T X$$

Linear Regression

Optimal solution

- In the form of matrix computation

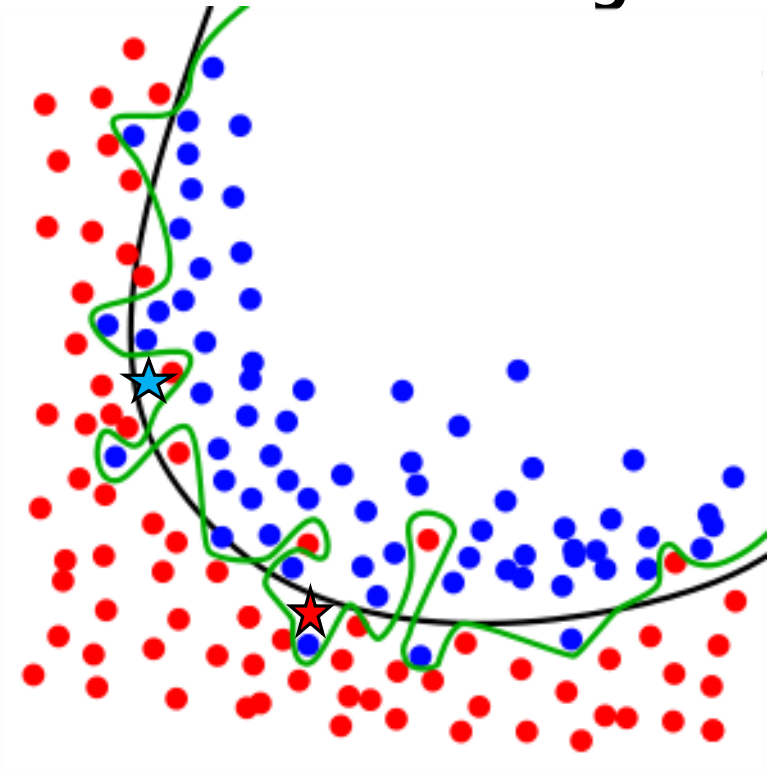
$$\hat{\boldsymbol{\beta}} = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

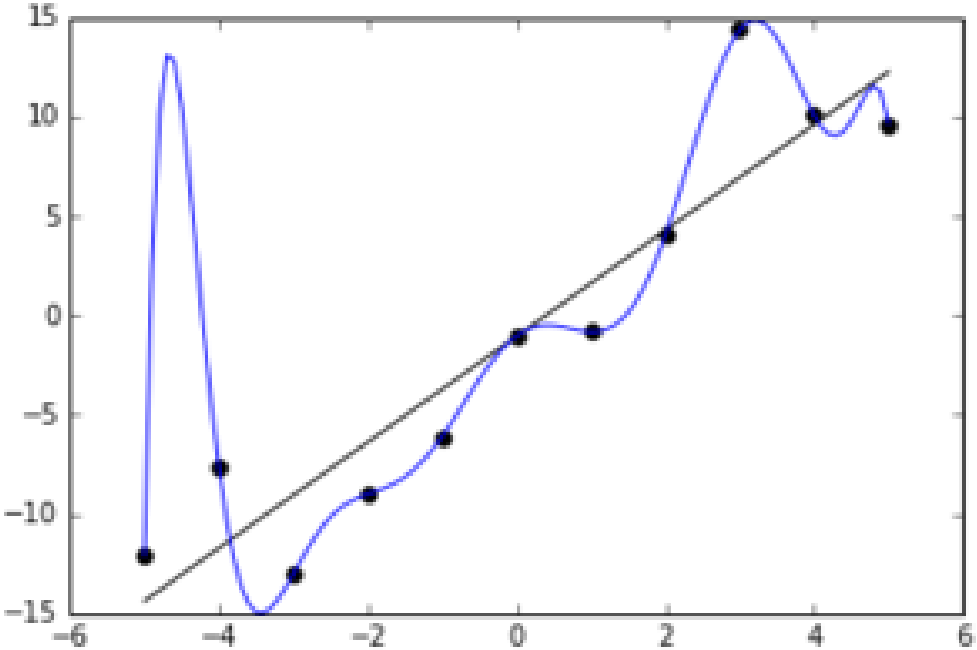


Closed form solution for the regression coefficient

Overfitting

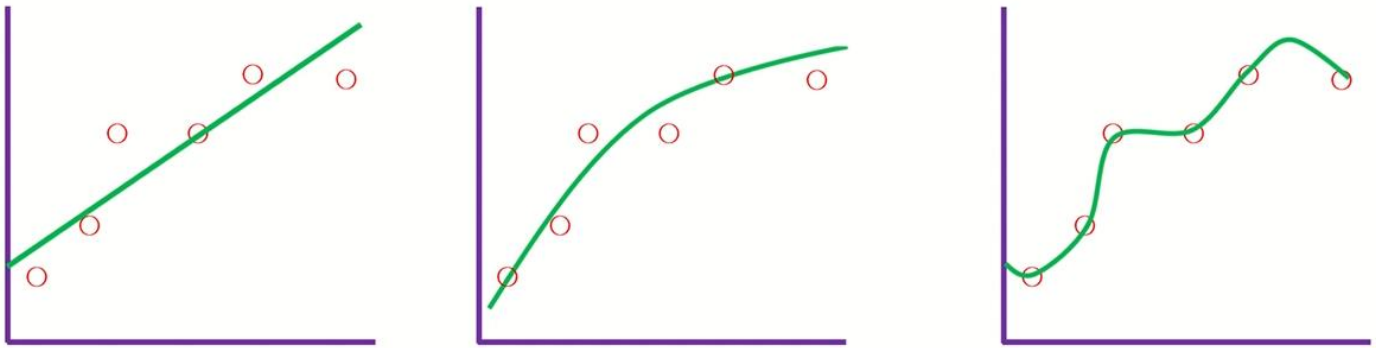
- Result of a training that makes a model fit training data too closely or exactly.
- Perfect on training data, but poor on test data





What is a Good Model?

- **Three different regression models based on the same data**

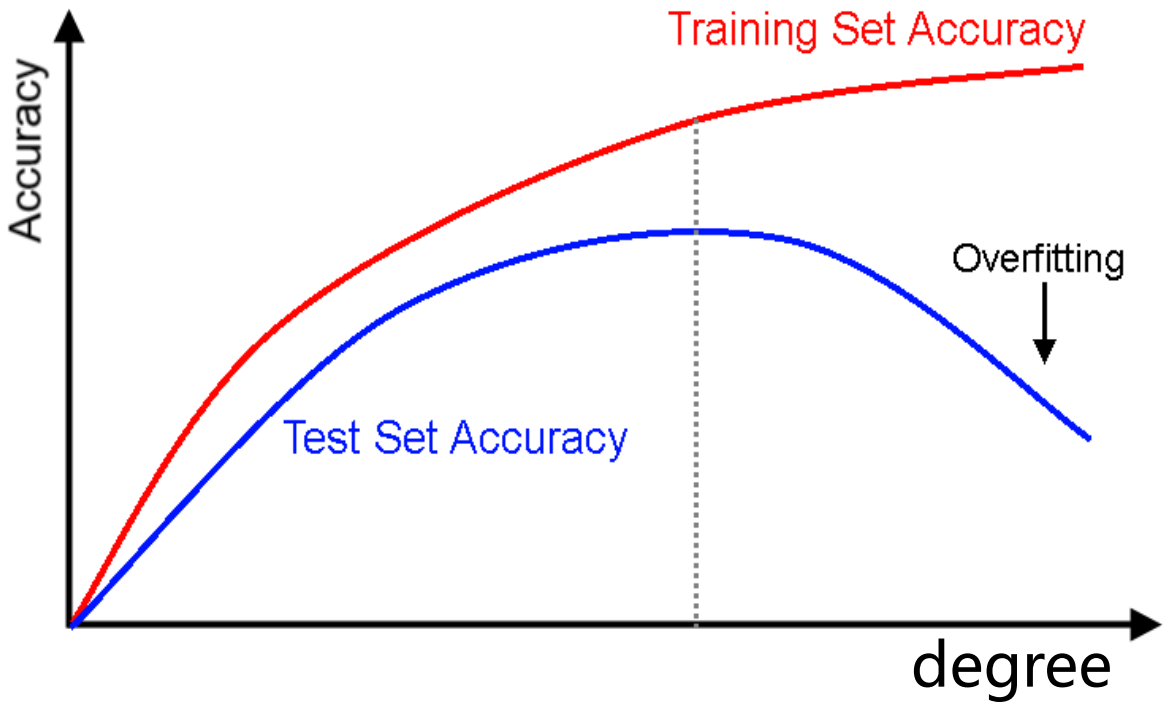


$\beta_0 + \beta_1 x$ $\beta_0 + \beta_1 x + \beta_2 x^2$ $\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4$

- Some models work well on training data
- But they might not work well on the unseen test data
- Which one is better?

What is a Good Model?

- A model with high complexity might cause **overfitting**



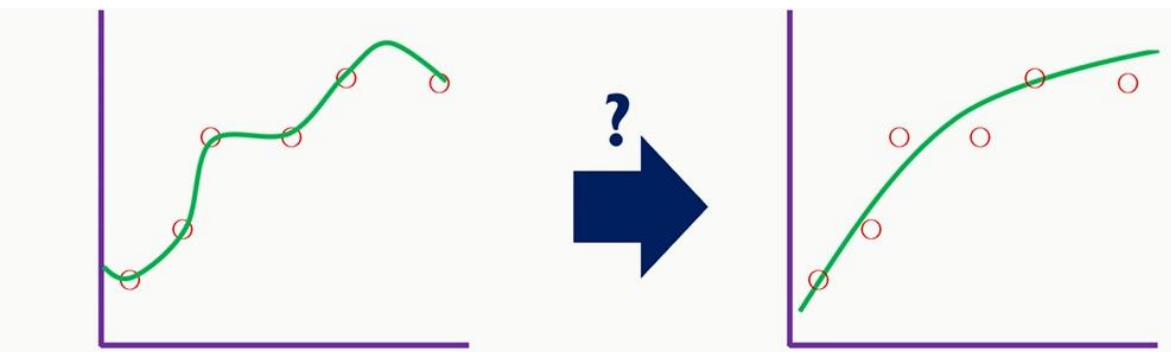
Regularization Methods

Adding regularizations to avoid overfitting

- How about adding another objectives to the original cost function?

$$\min_{\beta} \sum_{i=1} (y_i - \hat{y}_i)^2 + 5000{\beta_3}^2 + 5000{\beta_4}^2$$

- Then, model tries to… $\beta_3 \approx 0$ $\beta_4 \approx 0$



$\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4$ $\beta_0 + \beta_1 x + \beta_2 x^2$

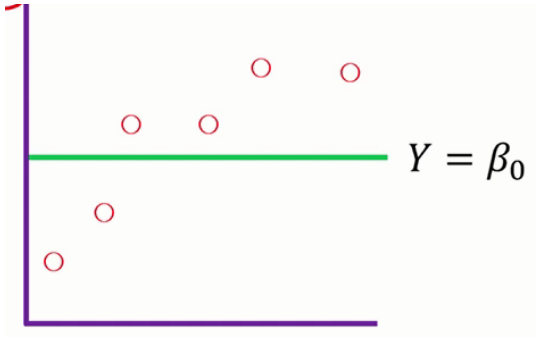
Regularization Methods

Adding regularizations to avoid overfitting

- Adding another objectives to the original cost function

$$L(\beta) = \min_{\beta} \underbrace{\sum_{i=1} (y_i - \hat{y}_i)^2}_{\text{(1) Training accuracy}} + \lambda \underbrace{\sum_{j=1}^{p} \beta_j^2}_{\text{(2) Generalization accuracy}}$$

λ very big $\rightarrow$ $\beta_1 \approx 0,\ \beta_2 \approx 0,\ \beta_3 \approx 0,\ \beta_4 \approx 0$



λ very small $\rightarrow$

$$\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4$$

Regularization Methods

- **Ridge Regression**
  - It adds **L2-norm** of model parameters

$$\hat{\beta}^{ridge} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} (y_i - x_i\beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right\}$$

Regularization Methods

- **There is a closed form solution for the ridge regression**

$$Q(\beta) = (y - X\beta)^T\ (y - X\beta) + \lambda\beta^T\beta$$

$$= y^T y\ - 2\beta^T X^T y + \beta^T\left(X^T X + \lambda I_p\right)\beta$$

$$\frac{\partial}{\partial\beta} Q(\beta) = -2X^T y + 2\left(X^T X + \lambda I_p\right)\beta = 0$$

$$\hat{\beta}^{ridge} = \left(X^T X + \lambda I_p\right)^{-1} X^T \mathrm{y}$$

$$\hat{\beta}^{LS} = \left(X^T X\right)^{-1} X^T \mathrm{y}$$

$$\beta_1, \beta_2, \ldots, \beta_p$$

Regularization Methods

- **Lasso (Least absolute shrinkage and selection operator)**
  - It adds **L1-norm** of model parameters

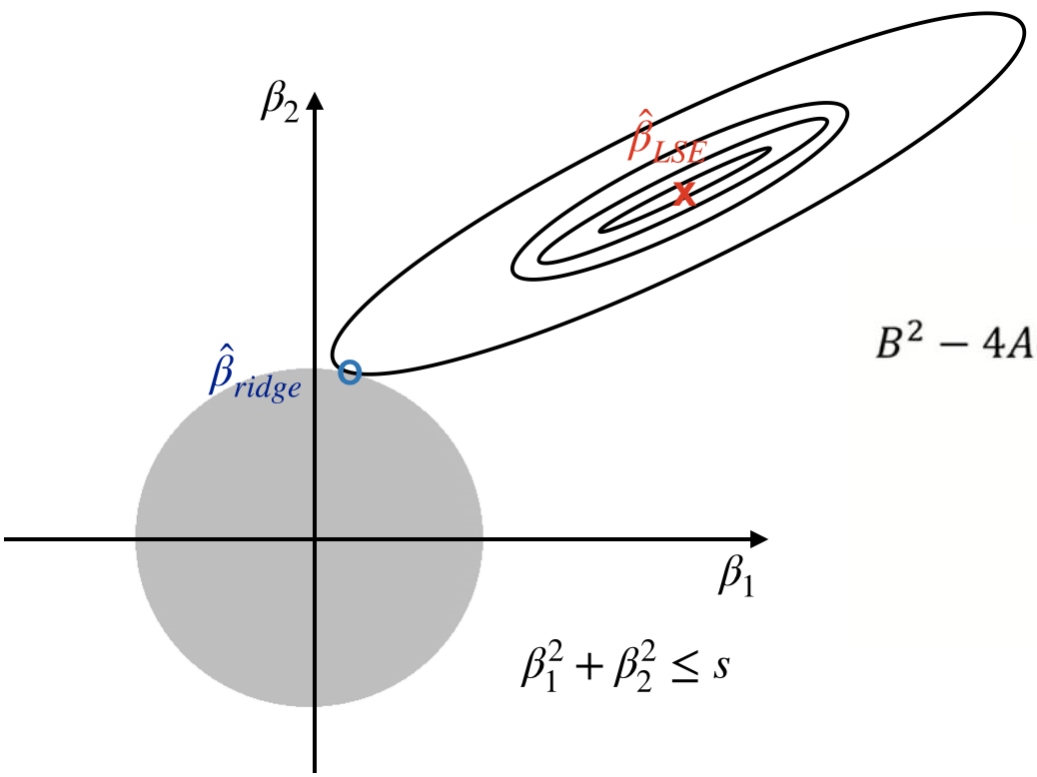
$$\hat{\beta}^{lasso} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} (y_i - x_i\beta)^2 + \lambda \sum_{j=1}^{p} \boxed{|\beta_j|} \right\}$$

  - There is no closed-form solution

Regularization Methods

❑ Ridge VS Lasso

- ❑ Lasso is able to "discard" less-important input features



$$MSE(\beta_1, \beta_2) = \sum_{i=1}^{n} (y_i - \beta_1 x_{i1} - \beta_2 x_{i2})^2$$

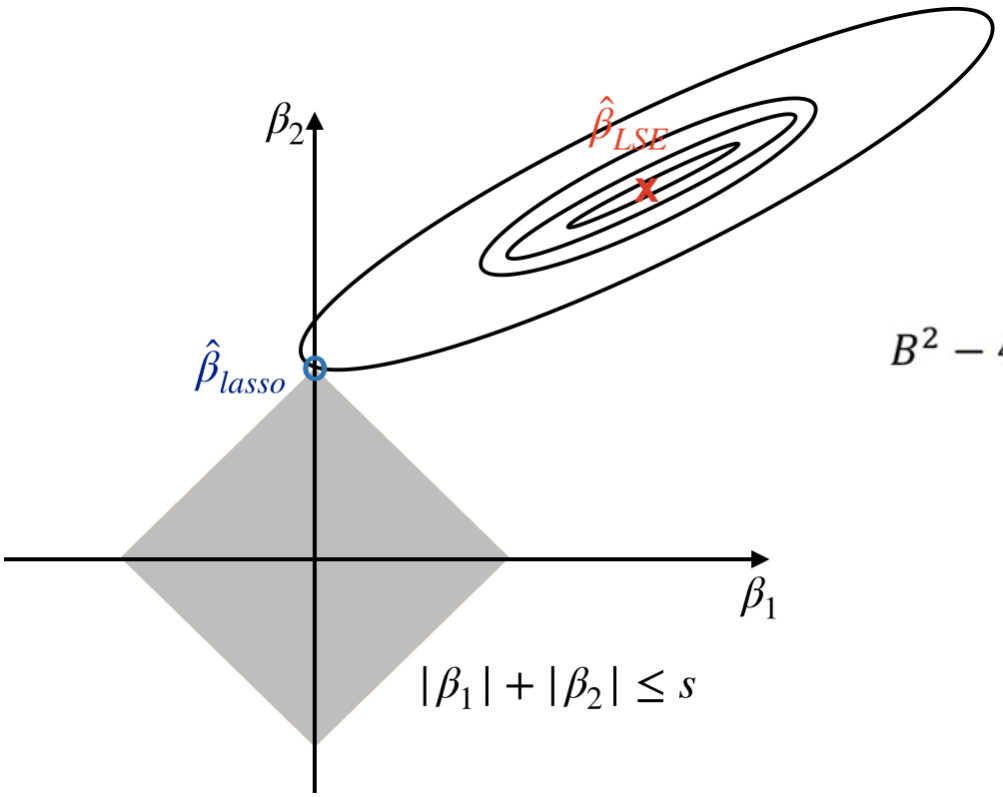
$$= A\beta_1^2 + B\beta_1\beta_2 + C\beta_2^2 + D\beta_1 + E\beta_2 + F$$

$$B^2 - 4AC = \left(2\sum_{i=1}^{n} x_{i1}x_{i2}\right)^2 - 4\sum_{i=1}^{n} x_{i1}^2 \sum_{i=1}^{n} x_{i2}^2$$

$$= 4\left\{\left(\sum_{i=1}^{n} x_{i1}x_{i2}\right)^2 - \sum_{i=1}^{n} x_{i1}^2 \sum_{i=1}^{n} x_{i2}^2\right\} < 0$$

Regularization Methods

- **Ridge VS Lasso**
  - Lasso is able to "discard" less-important input features



$$MSE(\beta_1, \beta_2) = \sum_{i=1}^{n} (y_i - \beta_1 x_{i1} - \beta_2 x_{i2})^2$$

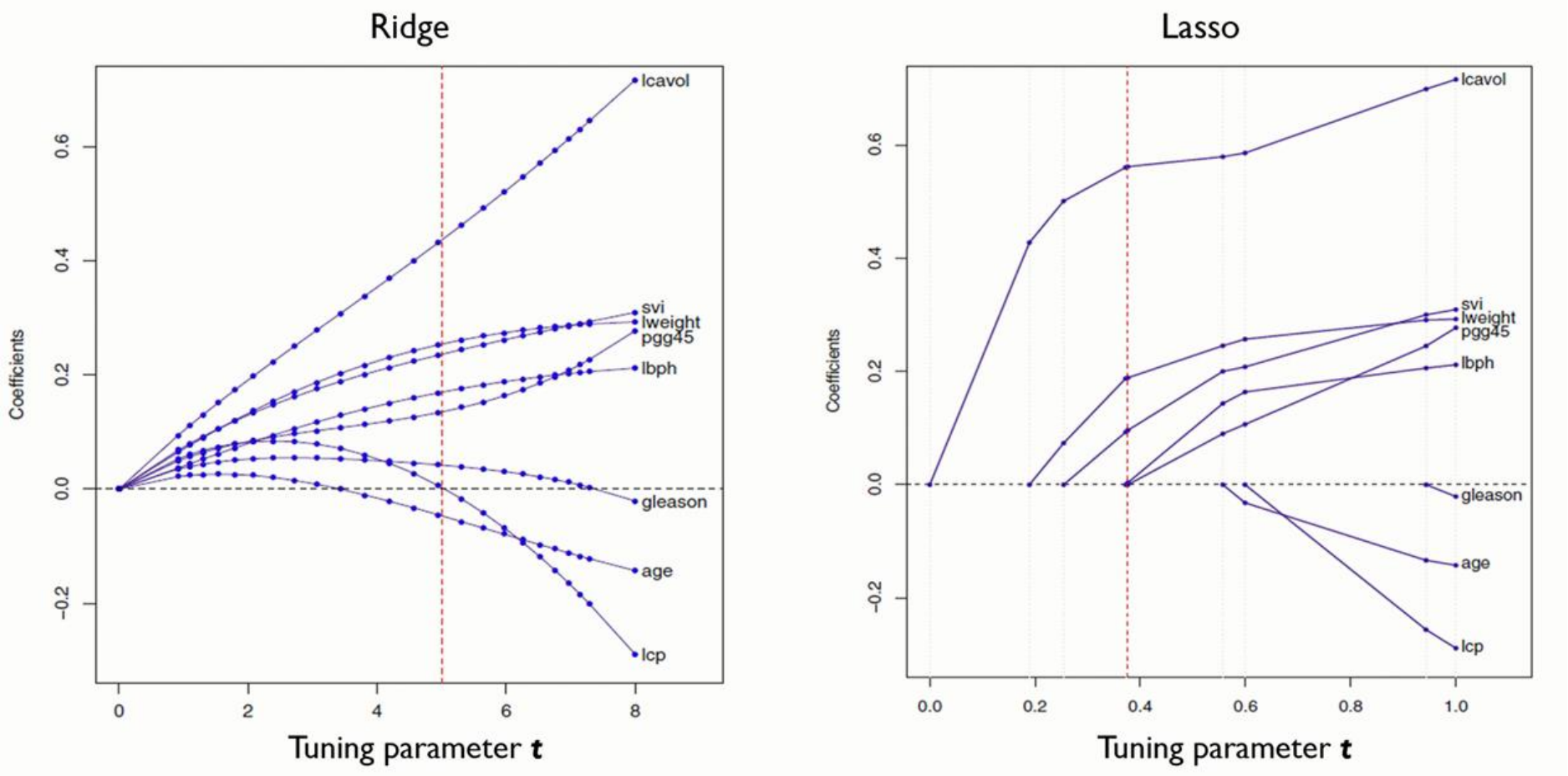
$$= A\beta_1^2 + B\beta_1\beta_2 + C\beta_2^2 + D\beta_1 + E\beta_2 + F$$

$$B^2 - 4AC = \left(2\sum_{i=1}^{n} x_{i1}x_{i2}\right)^2 - 4\sum_{i=1}^{n} x_{i1}^2 \sum_{i=1}^{n} x_{i2}^2$$

$$= 4\left\{\left(\sum_{i=1}^{n} x_{i1}x_{i2}\right)^2 - \sum_{i=1}^{n} x_{i1}^2 \sum_{i=1}^{n} x_{i2}^2\right\} < 0$$

Regularization Methods

- **Ridge VS Lasso**
  - Lasso is able to "discard" less-important input features



Regularization Methods

- **Elastic Net (Ridge + Lasso)**

$$\hat{\beta}^{enet} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} (y_i - x_i\beta)^2 + \lambda_1 \sum_{j=1}^{p} |\beta_j| + \lambda_2 \sum_{j=1}^{p} {\beta_j}^2 \right\}$$

- **More regularization methods**
  - Fused lasso
  - Group lasso
  - Grace
  - ….

Summary

- **(multiple linear) Regression**
  - Learn a linear relationship between a **target variable Y** and a set of **input features x_1, x_2, …, x_p**
  - Define a squared error cost function, and apply the closed-form solution to get the optimal parameters

- **Regularization methods**
  - Ridge, Lasso, etc…

- **Next topic: Logistic Regression, gradient descent optimization**

Thank You